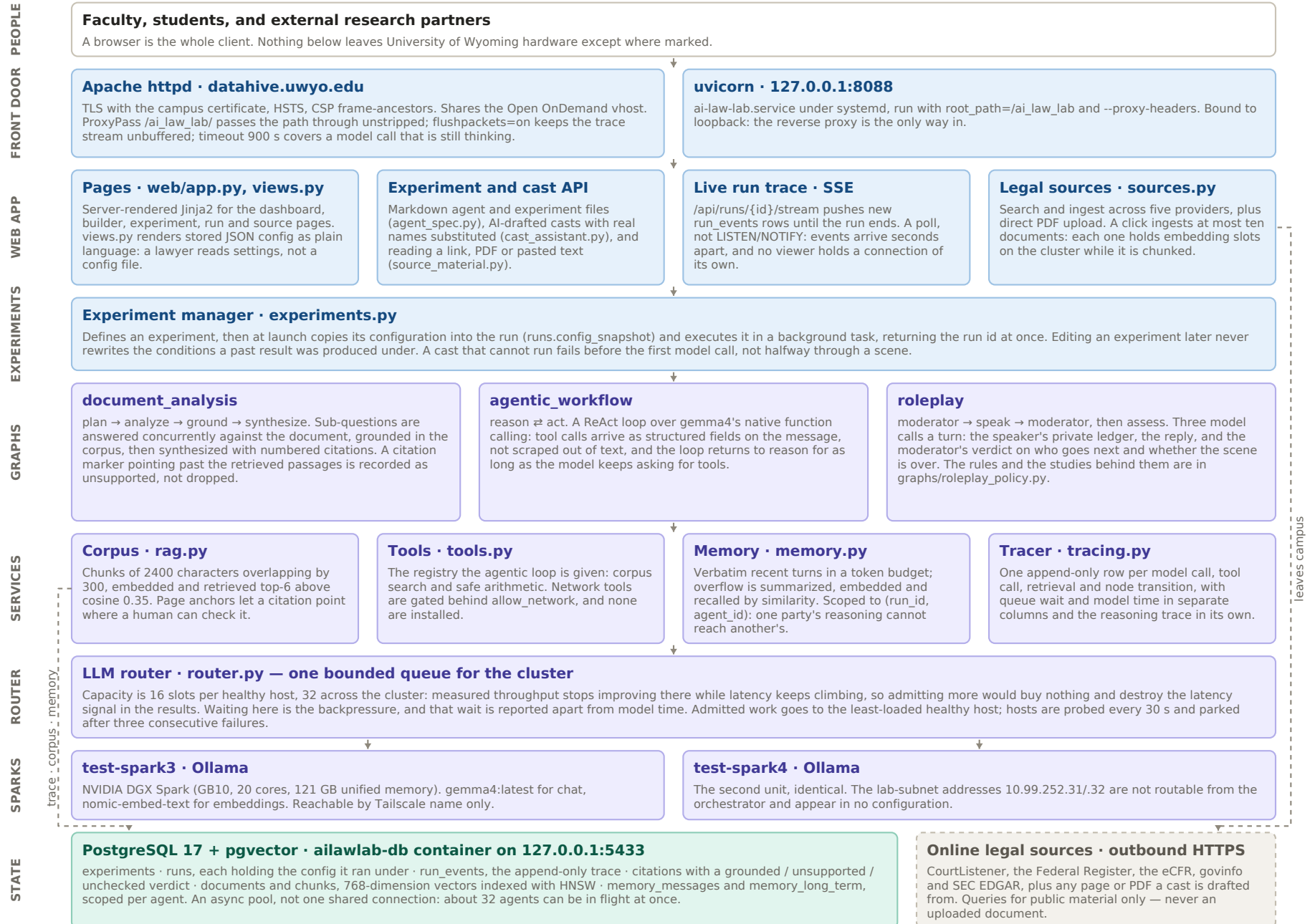


# AI Law Lab: System Architecture

How a request becomes a traced, reproducible experiment run

Jian Gong · AI Law Lab, University of Wyoming · Architecture note, September 2026. Describes the software at commit f689797.



## Reading the diagram

Each band is a tier, and the stack reads top to bottom in the order a request passes through it. Solid arrows are the path a launched run takes. Dashed arrows are the two edges that are not part of that path: what every tier persists, and the one direction in which traffic leaves the university. Colour carries the same meaning as on the portal itself -- blue for the application, purple for anything that consumes model capacity, green for durable state, grey for hardware and services outside the lab.

- **Two hops, one process.** Apache terminates TLS and proxies to a uvicorn worker bound to loopback. There is exactly one application process; concurrency inside it is asyncio, not workers, which is why a single bounded queue can speak for the whole cluster.
- **The orchestration band is three graphs, not three services.** They share the router, corpus, memory and tracer beneath them, and differ only in their nodes.
- **Everything narrows at the router.** Every model call in every mode, including embeddings during ingestion, passes through one admission point. That is what makes a timing number from one run comparable with a timing number from another.

## 1 • Front door

The lab is served at [https://datahive.uwyo.edu/ai\\_law\\_lab/](https://datahive.uwyo.edu/ai_law_lab/) from the same Apache vhost as the Open OnDemand portal. Three details in that configuration are load-bearing. The proxy passes the mount path through **unstripped**, because the application runs with `root_path=/ai_law_lab` and strips the prefix itself; proxying to a bare root would 404 its `/static` mount. `flushpackets=on` keeps the live trace stream from being buffered into uselessness. And `timeout 900` covers a model call that emits nothing while it is thinking -- with the default the browser would lose a role-play turn mid-generation.

X-Forwarded-Proto is set on the location and uvicorn runs with `--proxy-headers`: without both, the application would build pages from an http-scheme request and emit `http://` absolute URLs on an HSTS-pinned site. The unit file deliberately has no `EnvironmentFile` -- `.env` holds JSON whose inner quotes systemd's parser strips -- so pydantic-settings reads it directly, relative to the working directory.

## 2 • Web application

The interface is server-rendered Jinja2 with small amounts of JavaScript for the cast builder, not a single-page application. `views.py` exists so that what a page shows is a plain-language rendering of stored settings rather than the stored settings: an experiment page says how long a role-play is likely to take and shows each cast member as a card, because the people configuring experiments here are lawyers.

A run's trace reaches the browser over server-sent events. The endpoint polls `run_events` once a second rather than using `LISTEN/NOTIFY`: events arrive seconds apart, so a one-second poll is well inside budget and avoids holding a dedicated database connection open for every viewer.

## 3 • Experiment manager

`experiments.py` is the seam between the web tier and the graphs. Launching copies the experiment's configuration into `runs.config_snapshot` and only then starts work, so editing an experiment afterwards cannot rewrite the conditions a past result was produced under. Execution happens in a background task and the run id comes back immediately, which is what lets a browser watch an hour-long scene without holding a request open. A failure is recorded as a

status with its trace intact, not raised: a failed experiment is a result.

Before a role-play starts, the cast is checked. Without that, a duplicate agent id or a moderator pick matching no agent surfaces mid-run as a bare `StopIteration`, long after the cause was knowable. Lesser problems are written to the trace instead.

## 4 • Orchestration

Each mode is a LangGraph state graph over a typed state dictionary. Live services -- the router, tracer and corpus -- travel in `config.configurable` rather than in state, because state is checkpointed and connections are not serializable.

| Mode              | Nodes                                      | What ends it                                                                                 |
|-------------------|--------------------------------------------|----------------------------------------------------------------------------------------------|
| document_analysis | plan → analyze → ground → synthesize       | The pipeline is linear and runs once.                                                        |
| agentic_workflow  | reason ⇌ act                               | The model stops emitting tool calls, or the iteration ceiling is reached.                    |
| roleplay          | moderator → speak → moderator ... → assess | The moderator closes the scene, or max_turns is hit. Never before everyone has spoken twice. |

The recursion limit handed to a role-play is three times its turn ceiling plus twenty, since each turn costs a moderator hop and a speak hop.

## 5 • Shared services

- **Corpus** (`rag.py`) ingests a document once -- identity is a SHA-256 of its text -- chunks it at 2400 characters with 300 of overlap, embeds in batches so one large document cannot hold a cluster slot indefinitely, and retrieves the top six chunks above cosine 0.35 with page anchors attached.
- **Tools** (`tools.py`) is the registry the agentic loop is given: corpus search and a safe arithmetic evaluator. `allow_network` gates network-touching tools, and none are installed, so that switch currently changes nothing.
- **Memory** (`memory.py`) keeps a verbatim buffer inside a token budget and folds the overflow into model-written summaries that are embedded and recalled by similarity. Its scope is (`run_id`, `agent_id`): in a negotiation, a shared buffer would leak one side's private reasoning into the other's context and quietly invalidate the exercise.
- **Tracer** (`tracing.py`) writes one sequenced row per model call, tool call, retrieval and node transition. Queue wait and model time go in separate columns because the benchmark showed them diverging threefold under load; reported together, a run that was merely waiting looks like a run that was slow.

## 6 • Inference and the queue

Two DGX Spark units serve `gemma4:latest` through Ollama, reachable by Tailscale name only. The router in front of them is the piece of this architecture most worth explaining, because the obvious design is wrong. Benchmarking both hosts on 2026-07-23 found that fifty concurrent requests all complete -- nothing errors -- while aggregate throughput stops improving at about sixteen:

| Concurrency | Aggregate tok/s | p50 latency |
|-------------|-----------------|-------------|
| 1           | 18              | 0.66 s      |
| 16          | 160             | 1.87 s      |
| 32          | 131             | 3.29 s      |
| 50          | 160             | 5.22 s      |

Past sixteen, the extra requests are queueing inside Ollama, not running. Round-robin dispatch would push fifty requests at a host and then report the resulting latency as though it were model speed.

So the router keeps a single global semaphore sized to the healthy hosts' combined capacity -- sixteen slots each, thirty-two in total -- and queues everything else itself. Waiting there is the backpressure, and the time spent is what gets recorded as `queue_wait_ms`. Admitted work goes to the least-loaded healthy host, which self-balances better than round-robin when one agent is summarizing a two-hundred-page contract and another is emitting a one-line reply. Hosts are probed every thirty seconds, parked after three consecutive failures, and the global capacity is rebuilt when the healthy count changes.

One model property shapes the tiers above: `gemma4` reasons before answering by default. That reasoning is exactly what a research setting wants, and it is captured to its own column -- but it costs roughly an order of magnitude more output tokens on short answers. Nodes doing mechanical work (turn-taking, summarizing, planning) therefore pass `think=False`; nodes doing substantive legal reasoning pass `think=True`.

## 7 • State

One PostgreSQL 17 database with `pgvector`, in a container on loopback, holds everything a run produces. It is reached through an async pool rather than a single connection: the design this memory tier was ported from held one SQLite connection, which serializes hard under the roughly thirty-two concurrent agents this cluster sustains. Vector columns are 768-dimensional and indexed with HNSW, chosen over IVFFlat because the corpus grows incrementally and HNSW needs no retraining.

| Tables                                         | What they hold                                                                                                        |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>experiments, runs</code>                 | Definitions, and per-run snapshots of the configuration each result was produced under.                               |
| <code>run_events, citations</code>             | The append-only trace, and asserted citations with a grounded / unsupported / unchecked verdict.                      |
| <code>documents, chunks</code>                 | The corpus: text, page anchors, and embeddings.                                                                       |
| <code>memory_messages, memory_long_term</code> | Short-term buffers and summarized long-term memory, per (run, agent), with provenance back to a trace event or chunk. |

Everything is addressable by `run_id`, and every foreign key cascades from the run, so a result can be audited or removed as a unit.

## 8 • What leaves campus

Inference is local, so documents and queries stay on university hardware. The single exception is drawn dashed on the right of the diagram: outbound requests to public legal sources -- CourtListener, the Federal Register, the eCFR, govinfo and SEC EDGAR -- and to a page or PDF a cast is being drafted from. These carry a search term or a URL, never an uploaded document.

That outbound path is guarded, because this server can reach the Sparks, Postgres and the university network. Only http/https links whose host resolves exclusively to public addresses are fetched, and every redirect is re-checked the same way. A source is capped at 12,000 words before it reaches a prompt. Real people and private organizations in a source are renamed before a cast is drafted from it, and the same substitution is applied to the model's output: a draft invents bottom lines and confidential facts, and attaching those to a real company is the failure worth engineering against.

## Where each box lives

| Box on the diagram            | Source                                                                                                                  |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Pages, SSE, HTTP API          | src/ailawlab/web/app.py, web/views.py, web/templates/                                                                   |
| Experiment and cast API       | agent_spec.py, cast_assistant.py, source_material.py                                                                    |
| Legal sources                 | sources.py                                                                                                              |
| Experiment manager            | experiments.py                                                                                                          |
| Orchestration                 | graphs/document_analysis.py, graphs/agentic_workflow.py, graphs/roleplay.py, graphs/roleplay_policy.py, graphs/state.py |
| Corpus, tools, memory, tracer | rag.py, tools.py, memory.py, tracing.py                                                                                 |
| LLM router                    | router.py                                                                                                               |
| State                         | db.py, db/schema.sql, docker-compose.yml                                                                                |
| Front door                    | /etc/apache2/sites-available/ood-portal.conf,<br>/etc/systemd/system/ai-law-lab.service, .env                           |

Settings quoted throughout this note are defaults from config.py; every one is overridable by an AILAWLAB\_-prefixed environment variable.

## Rebuilding this note

```
uv venv /tmp/docenv
uv pip install --python /tmp/docenv/bin/python reportlab
/tmp/docenv/bin/python docs/architecture/build.py
```

The diagram is laid out in docs/architecture/build.py; the boxes are data at the top of that file. Re-run it after a change to the tiers, and re-run scripts/bench\_sparks.py after any hardware or Ollama change, since the numbers in section 6 are what the router's slot count is set from.